

# Guide Git

Bonnes pratiques — Formation DevCamp

## 1. Introduction

Git est un système de gestion de versions distribué qui permet de suivre l'évolution d'un projet, de revenir en arrière si besoin, et de travailler à plusieurs sur le même code sans écraser le travail des autres. Ce guide couvre les commandes essentielles utilisées pendant les ateliers de la formation, notamment pour versionner le code des agents développés en Python.

## 2. Installation et configuration

Git s'installe via le gestionnaire de paquets de votre système (apt, brew, ou l'installateur officiel sous Windows). Une fois installé, configurez votre identité, utilisée pour signer vos commits :

```
git config --global user.name "Votre Nom"
git config --global user.email "vous@exemple.fr"
```

## 3. Notions de base

Un dépôt (repository) est un dossier suivi par Git. Un commit est une photographie de l'état du projet à un instant donné, accompagnée d'un message décrivant le changement. La zone de staging (index) permet de choisir précisément quels fichiers modifiés seront inclus dans le prochain commit, via la commande git add.

```
git init
git add fichier.py
git commit -m "Message décrivant le changement"
```

## 4. Créer une branche

Une branche permet de développer une fonctionnalité ou de corriger un bug sans affecter la branche principale (souvent nommée main). C'est la pratique recommandée avant toute modification non triviale.

Pour créer une nouvelle branche et basculer dessus immédiatement, deux syntaxes équivalentes sont couramment utilisées :

```
git checkout -b nom-de-la-branche

# équivalent, syntaxe plus récente :
git switch -c nom-de-la-branche
```

Pour lister les branches existantes et voir sur laquelle vous vous trouvez (marquée d'un astérisque) :

```
git branch
```

Pour revenir sur une branche déjà existante sans en créer une nouvelle, on omet l'option -b ou -c :

```
git checkout nom-de-la-branche
git switch nom-de-la-branche
```

## 5. Fusionner une branche

Une fois le travail terminé sur une branche, on la fusionne dans la branche principale avec git merge, après s'être replacé sur la branche de destination :

```
git switch main
git merge nom-de-la-branche
```

En cas de conflit (les mêmes lignes modifiées différemment sur les deux branches), Git marque les zones en conflit directement dans les fichiers concernés avec des balises <<<<<<, ===== et >>>>>>. Il faut

éditer manuellement ces fichiers pour choisir la version à conserver, puis valider la résolution avec `git add` suivi d'un nouveau commit.

## 6. Bonnes pratiques de commit

Un commit doit correspondre à un changement logique unique et porter un message clair, à l'impératif présent (« Ajoute la fonction `get_room` » plutôt que « ajouté » ou « ajout »). Évitez les commits fourre-tout qui mélangent plusieurs sujets sans rapport.

## 7. Fichier `.gitignore`

Le fichier `.gitignore` liste les fichiers et dossiers que Git doit ignorer : environnements virtuels, fichiers de configuration contenant des secrets (comme les clés d'API dans un fichier `.env`), fichiers compilés, ou caches temporaires. C'est une bonne pratique systématique dès la création d'un nouveau projet.

## 8. Workflow recommandé pour les ateliers

Pour les projets d'agent développés pendant la formation, le workflow recommandé est le suivant : une branche `main` qui contient toujours une version fonctionnelle, une branche par nouvelle fonctionnalité (par exemple `ajout-outil-meteo`), et une fusion dans `main` uniquement une fois la fonctionnalité testée.